AceOS College Project Report

Asset Compliance and Equipment Operations System

Java OOP and RDBMS college application

v2.0.0 college freeze through V005

13 October 2026

This report documents the v2.0.0 local college freeze and preserves the earlier v1.0.0 report. All 89 configured checks passed; Task 6 was closed at Smit’s request. GitHub publication is separate.

Component	Verified configuration
Java and desktop	Java 24 | JavaFX 24.0.2
Database and access	MySQL 8.0.46 | JDBC Connector/J 26.7.0
Build and password library	Maven 3.9.16 | Bouncy Castle 1.86
Schema	V000 baseline plus COMPLETE V001-V005 | 16 tables
Release status	Local v2.0.0 freeze | v1.0.0 preserved

Contents

Section	Page
1 Project context and objectives	3
2 Final scope and users	3
3 Architecture and implementation	3
4 Lifecycle and correction workflow	4
5 Object oriented design	4
6 Relational design and normalization	5
7 Transactions and consistency	5
8 Validation security and recoverable UI	5
9 Verification and results	6
10 Setup and reproducibility	6
11 Limitations future scope and conclusion	7
12 Source references	7
Appendix A Entity relationship views	8
Complete foreign key relationships	11
Appendix B Complete data dictionary	12

1 Project context and objectives

AceOS records institutional equipment, receives requester complaints, assigns corrective repairs, preserves submitted work and administrator reviews, and tracks inspections and preventive schedules. A campus setting motivates the examples; this is an independent academic prototype, not an official institutional deployment or a proven operational installation.

The project demonstrates Java object modelling, state transitions, layered responsibilities, relational keys and constraints, JDBC persistence, transactions, role-scoped data access and recoverable desktop forms. Its main result is a complete complaint-to-verified-repair flow, including returning unsatisfactory work for correction without losing earlier evidence.

2 Final scope and users

All delivered functionality remains included: accounts/login/session safeguards; asset registration/profiles; complaints/comments/review reasons; withdrawal/reopening; assigned repairs and immutable correction history; workload/dashboards; inspections/independent safety holds; preventive schedules; and connected form recovery. The feature boundary is closed, not the final release process.

Role	Main responsibility	Important boundary
Requester	Submit as self and follow own complaints/history	No assignment verification or inspection management
Technician	Start/complete assigned repairs and inspections; correct returned work	No administrator return/verification; no reassignment after start
Administrator	Manage identities/assets; review/assign/return/verify; manage schedules/holds	Cannot resubmit technician correction or bypass independent evidence rules

Accounts are administrator-created. Roles use an enum and session-derived identity. Lists and exact detail reads are scoped before reconstruction; a hidden button is not the security boundary.

3 Architecture and implementation

JavaFX views call AceOsService; the service validates session/role/ownership and coordinates a transaction; repositories execute prepared SQL; MySQL enforces references and constraints. Domain objects express lifecycle changes. Security helpers provide authentication/session validation and the SQL scope. FxTasks performs database work on background virtual threads and returns callbacks to the JavaFX thread.

Layer	Main examples	Responsibility
Desktop UI	WorkspaceView RepairFlowView AssetProfileView	Forms navigation feedback and background coordination
Service	AceOsService	Authorized application operations transaction and error boundary
Model	Complaint WorkOrder MaintenanceRecord and record projections	Domain transitions and typed data
Persistence	RepairReviewRepository AvailabilityRepository and JDBC repositories	SQL mapping conditional writes and evidence reads
Security and transaction	Authentication Session Scope Transactions	Identity scopes resource ownership and commit rollback

The administrator sidebar has seven tabs: Dashboard, Assets, Complaints, Work orders, Inspections, Preventive schedules and Accounts. Creation/details stay within the parent route; My account is in the header. Navigation protects unsaved forms and prevents leaving/re-entering during a save.

4 Lifecycle and correction workflow

The normal complaint path is SUBMITTED to ACCEPTED to WORK_ORDER_CREATED to RESOLVED. Acceptance/rejection need an atomic recorded reason. A requester may withdraw an own SUBMITTED complaint to WITHDRAWN; requester/admin may reopen allowed WITHDRAWN records, and an administrator may reopen REJECTED records. Reopening requires a non-retired asset and no linked repair and returns to review without changing the original identity, priority, description or report time.

Repair progress is ASSIGNED to IN_PROGRESS to COMPLETED to VERIFIED. Administrator verification resolves its complaint. A completed unverified repair may instead become RETURNED with a mandatory administrator reason. The same technician submits fresh correction evidence and returns progress to COMPLETED; the administrator reviews that exact latest submission. Cycles may repeat. Verified work cannot return, and reassignment is allowed only before initial start.

Each work order retains its original assignment/start/first-completion time. Every correction is a new numbered maintenance record, and every return/verification reviews an exact submission. Earlier text is never overwritten. Current and accepted submission references include the work-order identity, preventing cross-work evidence. Historical missing authors/reviews remain explicitly unknown rather than fabricated.

Availability follows retired, then manual/safety hold, then any started-unverified repair, then active. Returning, resubmitting or completing does not clear availability restrictions. A verified repair closes only its own complaint; overlapping repairs and independent holds remain authoritative. Passing an inspection never automatically clears an earlier failed-inspection hold. Preventive completion remains date plus frequency and creates no work/history record.

5 Object oriented design

Encapsulation is visible in private WorkOrder/Complaint state and lifecycle methods. Services add authorization and persistence invariants. Composition links WorkOrder to Complaint/User, Complaint to Asset/reporter, and MaintenanceRecord to its work order. These are associations, not a claim that Java object deletion owns database history.

Abstraction separates UI/service/repository concerns. JDBC exposes Connection, PreparedStatement and ResultSet interfaces. Transactions implements AutoCloseable and supplies a borrowed Connection proxy so only the outer owner commits/closes. Generic Callable and ServiceOperation contracts separate an operation from background execution/authorization coordination.

Inheritance is used by JavaFX view classes: WorkspaceView extends BorderPane, RepairFlowView extends ScrollPane and DashboardView extends VBox. The domain does not invent requester/technician/administrator subclasses; UserRole represents those roles. Test preview services override AceOsService methods for controlled UI fixtures. JDBC driver interface implementations and that test substitution demonstrate polymorphism.

Overloaded constructors support new/reconstructed models, enums protect typed lifecycle/role values, and records describe bounded read projections. Records referencing mutable entities are not automatically deeply immutable. Asset/User legacy setters exist; public application mutations still belong to guarded services. See the separate OOP mapping for exact class references.

6 Relational design and normalization

The final schema has 16 tables: the original seven domain tables, accounts/audit/coordination/migration infrastructure, lifecycle/hold history, complaint notes and repair reviews. Primary keys identify records; foreign keys preserve links; unique constraints enforce code/login identities, one repair per complaint, submission numbering and one review per submission. Checks restrict persisted states and evidence shape; service checks enforce ownership, eligibility and temporal evidence.

Entities are decomposed by responsibility to avoid repeating asset/requester/work data in each record. Author snapshots in repair submissions/reviews deliberately preserve historical names/roles; assets.status is a derived projection persisted by transactionally recalculated availability. These are deliberate historical/projection choices, so the project does not claim every stored value is independent or that a formal whole-schema normal-form proof was performed.

Complaint notes store author ID and role at posting, with displayed names joined from current users; repair submissions/reviews additionally snapshot names. Audit target_id is event-specific and has no universal FK. security_gate, operator_setup and schema_migrations are infrastructure tables, not user-facing business entities. The dictionary documents every column, default, nullability, key and rule; the ER appendix includes every FK.

7 Transactions and consistency

AceOsService.execute owns authorization, writes and audit within Transactions. READ COMMITTED avoids a stale snapshot after waiting for the local security gate; asset locks coordinate availability. The borrowed JDBC connection ignores nested repository commit/close attempts so an operation cannot partly commit. An expected-state write must affect one row; any failure prevents commit and rolls back.

Return inserts an immutable review/reason, changes COMPLETED to RETURNED for the expected submission, recalculates availability and audits together. Resubmission inserts the next record, conditionally advances state/current pointer and audits without resetting old timestamps. Verification inserts the review, records the accepted submission/verification time, resolves the complaint and recalculates availability together. Audit failure therefore cannot leave half a lifecycle transition.

Failed-inspection completion and creation of its independent hold are atomic. Hold clearance separately checks a later passing reinspection or verified repair on the same asset. The accepted correction description is used for repair evidence, never returned text. Schema migrations are different: MySQL DDL is not treated as rollback-safe and interrupted migrations require reviewed restoration/repair.

8 Validation security and recoverable UI

Authentication uses salted Argon2id. Password replacement accepts 15-128 Unicode characters; temporary credentials expire after 24 hours. Five failed logins within 15 minutes impose a 15-minute cooldown; sessions expire after 30 idle minutes. Authentication versions invalidate sessions after password/reset/role/deactivation changes. Outstanding work blocks unsafe technician changes; self/last-administrator safeguards remain.

Comments/review/return reasons have nonblank 2000-Unicode-character bounds. Other maintenance/inspection text uses existing 5000-Java-string-character validation. Prepared statements bind inputs. Domain/service validation and SQL constraints are complementary. New submission/note/review tables have runtime SELECT/INSERT only, with no edit/delete path.

Background forms freeze write inputs and suppress duplicates. Failed writes retain drafts and require fresh progress before retry. Successful writes remain explicit through a failed follow-up read; refresh does not repeat the save. Unrelated/closed drafts remain copyable. Stay is the default departure choice; saving blocks navigation/re-entry and the guard stores no draft/password copies.

The trusted local client is the authorization boundary. Someone with database credentials or a modified client can bypass Java checks. Remote multi-user enforcement and production deployment are outside this college project.

9 Verification and results

Final v2.0.0 verification passed 89 configured checks: 46 database, 3 password/session and 40 JavaFX, with zero failures/errors/skips, and built the JAR in the same run. Repair return previously passed ten distinct database and six distinct JavaFX focused checks across implementation and verification, with zero failures/errors/skips in final focused runs. Packaging passed afterward with tests skipped in that packaging step. Controlled layouts/actions were reviewed at 800 by 520 and 1280 by 800. Repeated runs overlap these totals; earlier chunk counts also overlap and are not a single final full-suite count.

Database coverage includes roles/ownership/reset-invalidated sessions, correction cycles/history, stale exact-submission/return-review guards, competing actions/rollback, cross-work references, immutable runtime permissions, privacy/counts, independent holds/overlapping repairs and accepted evidence. Migration checks cover real V004 upgrade, fresh V000 through V005 setup, preflight inconsistency, rerun, STARTED/checksum mismatch, weakened rules and missing references.

UI coverage includes role controls, history labels, dashboard correction filter, associated labels/focus/visible actions, drafts, duplication/save/departure protection, verification feedback/read recovery and copyable closed drafts. Smit confirmed all five live repair-return walkthrough steps worked. He also confirmed earlier role/workload/withdrawal/reopening and remaining keyboard/screen-reader/reset/invalidation/feedback checks. These human reports are distinct from automated fixtures; individual earlier inspection/preventive edge cases were not separately recorded live.

Before V005, both isolated backups restored with all 15 table checksums matching. Migration preserved all 14 original non-migration tables/fields and V001-V004 metadata, inventing no historical actor/review rows. Original aceos and v1.0.0 stayed intact. Documentation updates do not justify rerunning unchanged workflows; final configured checks and local packaging are complete for v2.0.0.

10 Setup and reproducibility

The verified stack is Java 24, JavaFX 24.0.2, MySQL 8.0.46, Connector/J 26.7.0, Bouncy Castle 1.86 and Maven 3.9.16 on Windows. The existing private isolated instance uses loopback port 13316. Use fresh aceos_next_dev and disposable aceos_next_test, separate operator/runtime identities and environment-provided credentials. Existing activated development accounts need no reset/bootstrap.

On an empty selected schema execute V000__baseline.sql, then explicitly run com.aceos.database.Migrations with private migration variables. Grant the table-scoped runtime permissions in the setup guide; maintenance_records, complaint_notes and repair_reviews are SELECT/INSERT only, schema_migrations is read-only, and runtime cannot access operator_setup. Bootstrap the first administrator privately in an interactive console; run mvn javafx:run with runtime credentials.

mvn verify requires a disposable test URL plus runtime/test-operator credentials for database checks. ACEOS_UI_TEST=true enables graphical checks. Without those conditions, skipped checks are not correctness evidence. Tests may rebuild only disposable fixtures. database/demo_reset.sql selects original aceos and belongs to v1.0.0; never use it to prepare the current build or valuable data. Application startup verifies schema; it does not migrate automatically.

Close all AceOS windows before schema changes, back up and prove restoration, then migrate only the intended isolated schema. Keep dumps/configuration private. Maven now identifies v2.0.0; follow the preserved v1.0.0 tag for reproducing that older baseline. A standalone JAR is not claimed to bundle JavaFX/MySQL or private configuration.

11 Limitations future scope and conclusion

The final feature scope is sufficient for a local OOP/RDBMS college demonstration. There is no cancellation or return of verified repairs, no correction reassignment, no preventive work/history extension, and no automatic hold release. Additional notifications/calendar/attachments/QR/import-export/reporting, capacity/deadline/escalation, locations, commercial/inventory/vendor/integration and personalization features are future ideas, not college completion obligations. Deployment and pre-deployment preparation are excluded.

The final result is a coherent role-based application with traceable repair correction, relational integrity and transactional consistency. Tasks 1-5 are complete. Task 6 was closed at Smit's request, without generated demo/viva artifacts. Task 7 final checks/package/local freeze is complete as v2.0.0; the frozen v1.0.0 baseline remains preserved.

12 Source references

The report describes the completed v2.0.0 college application. Current source and documentation are maintained on main. The annotated v2.0.0 tag identifies the packaged source; the GitHub release provides the college package and source archive.

• pom.xml and src/main/resources/database/V000__baseline.sql through V005__repair_corrections.sql

• src/main/java/com/aceos/model/Complaint.java and WorkOrder.java

• src/main/java/com/aceos/service/AceOsService.java

• src/main/java/com/aceos/database/Transactions.java Migrations.java and Bootstrap.java

• src/main/java/com/aceos/security/Authentication.java Session.java Scope.java and Passwords.java

• src/main/java/com/aceos/repository/RepairReviewRepository.java AvailabilityRepository.java and FoundationRepository.java

• src/main/java/com/aceos/ui/WorkspaceView.java FxTasks.java NavigationGuard.java and view/RepairFlowView.java

• src/test/java/com/aceos/FoundationTest.java and UiSmokeTest.java

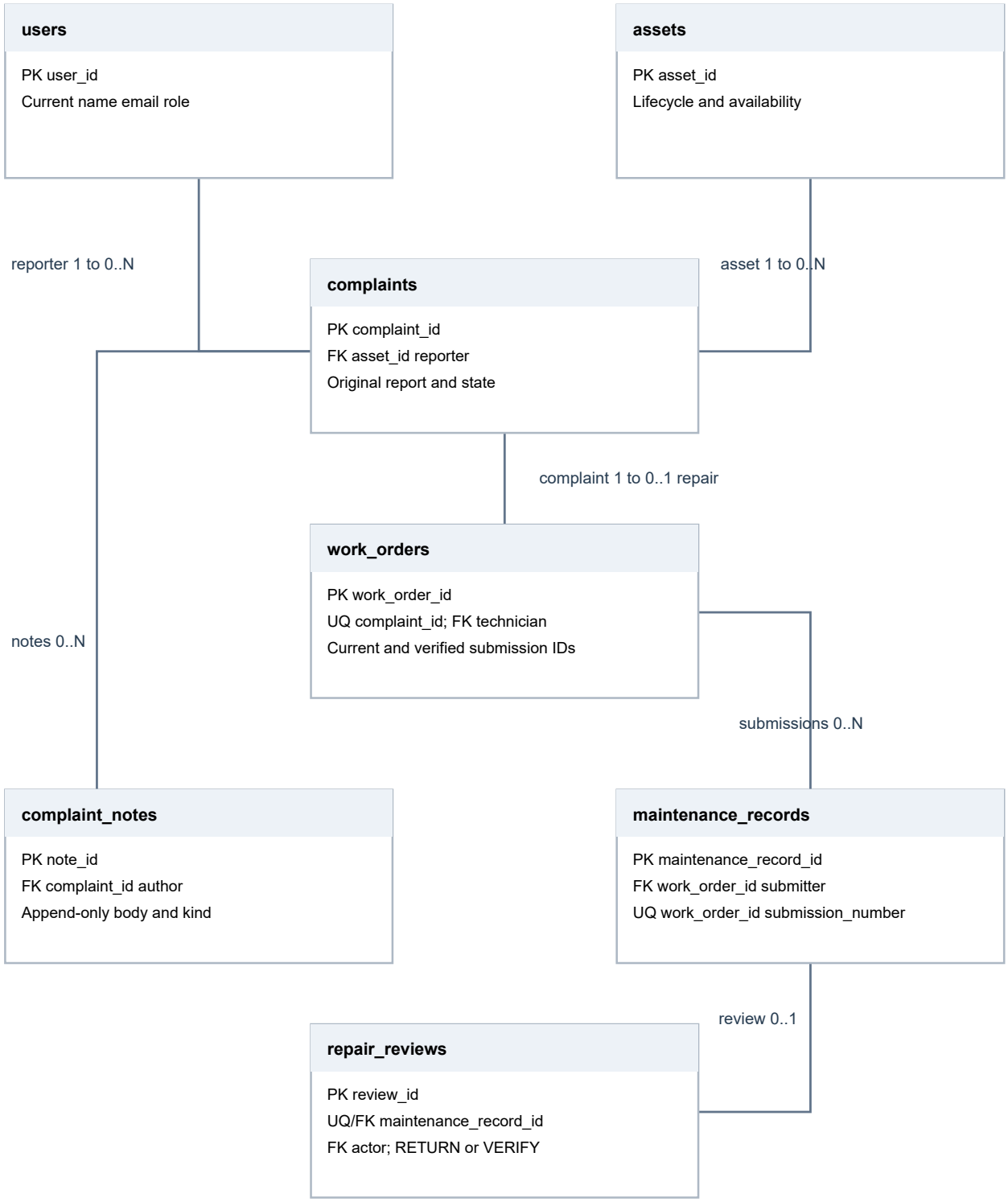
• docs/college-scope.md foundation.md oop-design.md transactions.md verification.md and data-dictionary.md

The project repository and preserved baseline are at https://github.com/SmitHitnalli/aceos. The Notion knowledge hub is a documentation companion, not the runtime database.

Appendix A Entity relationship views

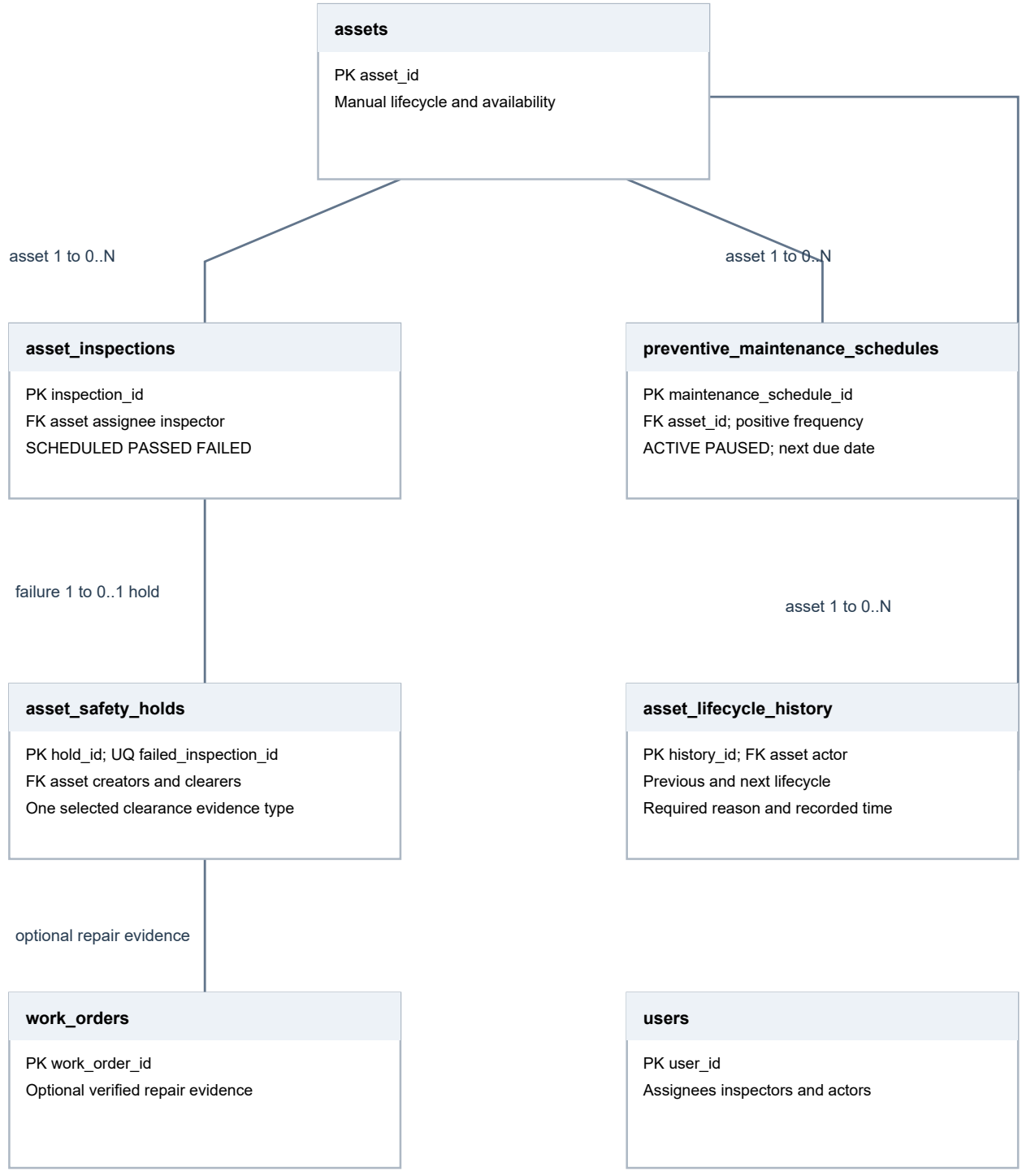
Three views show all 16 tables. Grouped actor/evidence links are enumerated in the complete FK table; the editable Mermaid map includes every column and relationship.

Complaint and repair relationships



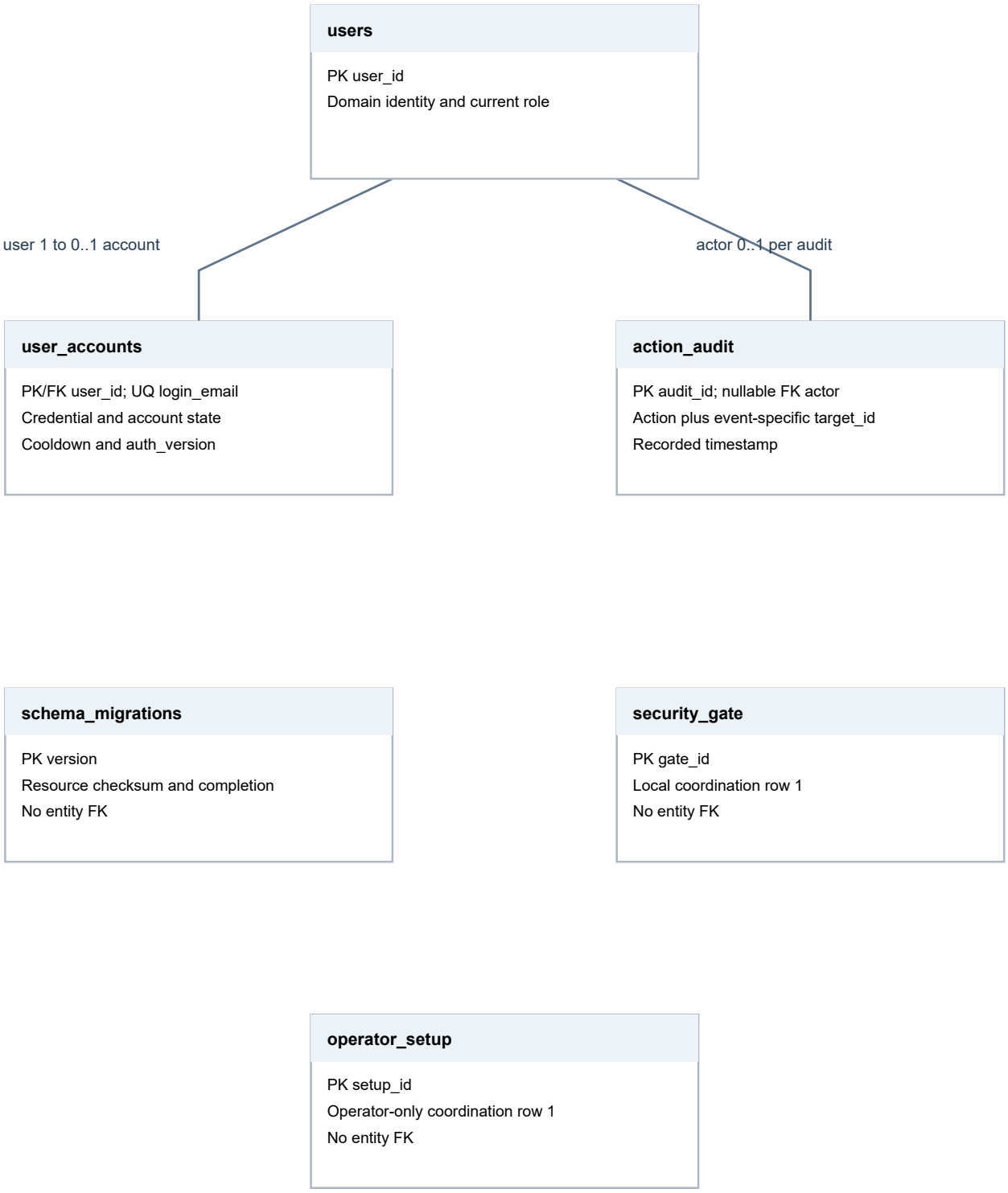
Actor FKs and same-work current/verified pointers are fully listed in the relationship map.

Availability inspections and preventive schedules



Holds also reference the asset, creating/clearing users and optional passing inspection.
All actor/evidence FKs are enumerated in the complete relationship map.

Accounts audit and infrastructure



target_id has no universal FK. Infrastructure rows are not linked to entities.

Complete foreign key relationships

Child	Fields	Parent	Referenced fields
action_audit	actor_user_id	users	user_id
asset_inspections	asset_id	assets	asset_id
asset_inspections	inspected_by_user_id	users	user_id
asset_inspections	assigned_to_user_id	users	user_id
asset_lifecycle_history	asset_id	assets	asset_id
asset_lifecycle_history	actor_user_id	users	user_id
asset_safety_holds	asset_id	assets	asset_id
asset_safety_holds	failed_inspection_id	asset_inspections	inspection_id
asset_safety_holds	created_by_user_id	users	user_id
asset_safety_holds	cleared_by_user_id	users	user_id
asset_safety_holds	passing_inspection_id	asset_inspections	inspection_id
asset_safety_holds	verified_work_order_id	work_orders	work_order_id
complaint_notes	author_user_id	users	user_id
complaint_notes	complaint_id	complaints	complaint_id
complaints	asset_id	assets	asset_id
complaints	reported_by_user_id	users	user_id
maintenance_records	work_order_id	work_orders	work_order_id
maintenance_records	submitted_by_user_id	users	user_id
preventive_maintenance_schedules	asset_id	assets	asset_id
repair_reviews	actor_user_id	users	user_id
repair_reviews	maintenance_record_id	maintenance_records	maintenance_record_id
user_accounts	user_id	users	user_id
work_orders	work_order_id, current_submission_id	maintenance_records	work_order_id, maintenance_record_id
work_orders	complaint_id	complaints	complaint_id
work_orders	technician_user_id	users	user_id
work_orders	work_order_id, verified_submission_id	maintenance_records	work_order_id, maintenance_record_id

Appendix B Complete data dictionary

All 16 current tables are included. PK denotes primary key, FK denotes foreign key, and UQ denotes membership of a unique tuple. SQL nullability is listed independently of conditional service/SQL requirements. MySQL BOOLEAN appears as tinyint(1). No application rows or credentials are included.

users

Domain identities and current roles; retained rather than deleted.

Field	SQL type	Null	Key	Meaning
user_id	int	NO	PK	Primary identifier.
name	varchar(100)	NO	-	Current display name.
email	varchar(255)	NO	UQ member	Domain email; unique in users.
role	varchar(20)	NO	-	Current REQUESTER, TECHNICIAN or ADMINISTRATOR role.

Roles are constrained; email is unique. Account/assignment safeguards are service rules.

Reported defaults: user_id = <none> (auto increment).

Unique tuple: email.

user accounts

Authentication state, normalized login, credential hashes, cooldown and session invalidation version.

Field	SQL type	Null	Key	Meaning
user_id	int	NO	PK, FK	Primary identity and users foreign key.
login_email	varchar(255)	NO	UQ member	Trimmed lowercase login identifier, unique separately from domain email.
password_hash	varchar(255)	YES	-	Encoded Argon2id hash, never plaintext; null before activation/when disabled.
state	varchar(25)	NO	-	Persisted account state; see table rules.
must_change	tinyint(1)	NO	-	Forced-password-replacement flag; BOOLEAN stored by MySQL as tinyint.
temporary_expires_at	timestamp	YES	-	Expiry of issued temporary credentials.
failed_attempts	int	NO	-	Persistent failed-login counter.
failure_window	timestamp	YES	-	Start of failure-counting window.
locked_until	timestamp	YES	-	Cooldown expiry; LOCKED is a derived UI state.
auth_version	bigint	NO	-	Incremented to invalidate existing sessions.

Persisted states are PENDING_ACTIVATION, ACTIVE and DISABLED. LOCKED is derived from locked_until; not a fourth stored state.

Reported defaults: state = PENDING_ACTIVATION; must_change = 1; failed_attempts = 0; auth_version = 1.

Unique tuple: login_email.

assets

Registered asset details, manual lifecycle and transactionally recalculated availability.

Field	SQL type	Null	Key	Meaning
asset_id	int	NO	PK	Primary identifier.
asset_code	varchar(50)	NO	UQ member	Unique human-readable asset code.
name	varchar(100)	NO	-	Current display name.
category	varchar(100)	NO	-	Asset category text.
location	varchar(150)	NO	-	Asset location text.
status	varchar(30)	NO	-	Current workflow/availability state; see table rules.
manual_lifecycle	varchar(30)	NO	-	Administrator-controlled ACTIVE, OUT_OF_SERVICE or RETIRED lifecycle.

Effective status is ACTIVE, UNDER_MAINTENANCE, OUT_OF_SERVICE or RETIRED. Manual lifecycle omits UNDER_MAINTENANCE. Effective status is a derived value persisted by recalculation.

Reported defaults: asset_id = <none> (auto increment).

Unique tuple: asset_code.

complaints

Requester report and current complaint lifecycle, preserving original identity and reported time.

Field	SQL type	Null	Key	Meaning
complaint_id	int	NO	PK	Primary identifier.
asset_id	int	NO	FK	Related asset reference.
reported_by_user_id	int	NO	FK	Original requester reference.
description	text	NO	-	Original reported problem.
priority	varchar(20)	NO	-	LOW, MEDIUM, HIGH or CRITICAL.
status	varchar(30)	NO	-	Current workflow/availability state; see table rules.
reported_at	datetime	NO	-	Original complaint report timestamp.

States: SUBMITTED, ACCEPTED, REJECTED, WORK_ORDER_CREATED, RESOLVED, WITHDRAWN. Closed complaints cannot receive comments. Reopening requires no linked work and a non-retired asset.

Reported defaults: complaint_id = <none> (auto increment); reported_at = CURRENT_TIMESTAMP.

work orders

One repair per complaint; assignment, progress and exact current/accepted submission links.

Field	SQL type	Null	Key	Meaning
work_order_id	int	NO	PK, FK	Primary identifier.
complaint_id	int	NO	FK, UQ member	Parent complaint reference.
technician_user_id	int	NO	FK	Assigned technician reference; reassignment only before start.
status	varchar(20)	NO	-	Current workflow/availability state; see table rules.
assigned_at	datetime	NO	-	Original assignment time.
started_at	datetime	YES	-	First start time; null before start.
completed_at	datetime	YES	-	First repair completion time; preserved during correction cycles.
verified_at	datetime(6)	YES	-	Final administrator verification time; null while unverified.
current_submission_id	int	YES	FK	Exact current submission; composite FK includes this work_order_id.
verified_submission_id	int	YES	FK	Exact accepted submission; composite FK includes this work_order_id.

States: ASSIGNED, IN_PROGRESS, COMPLETED, RETURNED, VERIFIED. complaint_id unique; started/first-completed timestamps retained across correction. Composite pointer FKs enforce same-work evidence; services enforce state coherence.

Reported defaults: work_order_id = <none> (auto increment); assigned_at = CURRENT_TIMESTAMP.

Unique tuple: complaint_id.

maintenance records

Immutable numbered repair submissions, including the original completion and corrections.

Field	SQL type	Null	Key	Meaning
maintenance_record_id	int	NO	PK, UQ member	Primary identifier.
work_order_id	int	NO	FK, UQ member	Parent repair reference.
action_taken	text	NO	-	Submitted work/correction description; earlier text is not overwritten.
recorded_at	datetime(6)	NO	-	Stored submission/note/review/audit recording time.
submission_number	int	NO	UQ member	Positive sequence within a work order; existing records migrate as 1.
submitted_by_user_id	int	YES	FK	Submitter reference; null for unknown migrated historical authors.
author_name	varchar(100)	YES	-	Name snapshot at submission/review; unknown legacy submission stays null.
author_role	varchar(20)	YES	-	Role snapshot at posting; repair submitter TECHNICIAN, reviewer ADMINISTRATOR.

UNIQUE(work_order_id, submission_number). Positive submission number; author snapshot is wholly unknown or contains a technician identity/name/role. Runtime SELECT/INSERT only.

Reported defaults: maintenance_record_id = <none> (auto increment); recorded_at = CURRENT_TIMESTAMP(6); submission_number = 1.

Unique tuple: work_order_id, submission_number.

Unique tuple: work_order_id, maintenance_record_id.

repair reviews

Append-only terminal administrator RETURN or VERIFY review of an exact submission.

Field	SQL type	Null	Key	Meaning
review_id	int	NO	PK	Primary identifier.
maintenance_record_id	int	NO	FK, UQ member	Exact reviewed submission reference.
kind	varchar(10)	NO	-	RETURN or VERIFY terminal review kind.
actor_user_id	int	NO	FK	Authenticated reviewing administrator reference.
author_name	varchar(100)	NO	-	Administrator name snapshot at review.
author_role	varchar(20)	NO	-	Administrator role snapshot at review.
reason	varchar(2000)	YES	-	Required explanation for lifecycle/RETURN; VERIFY review has null reason.
recorded_at	datetime(6)	NO	-	Stored submission/note/review/audit recording time.

One review per maintenance_record_id. kind RETURN requires nonblank reason up to 2000 Unicode characters; VERIFY has no reason. Reviewer role ADMINISTRATOR. Runtime SELECT/INSERT only.

Reported defaults: review_id = <none> (auto increment); recorded_at = CURRENT_TIMESTAMP(6).

Unique tuple: maintenance_record_id.

complaint notes

Append-only comments and required review/withdrawal/reopening reasons.

Field	SQL type	Null	Key	Meaning
note_id	bigint	NO	PK	Primary identifier.
complaint_id	int	NO	FK	Parent complaint reference.
author_user_id	int	NO	FK	Posting user reference; displayed complaint-note name joins current users.name.
author_role	varchar(20)	NO	-	Requester, technician or administrator role snapshot at posting.
kind	varchar(20)	NO	-	COMMENT or acceptance/rejection/withdrawal/reopening reason kind.
body	varchar(2000)	NO	-	Nonblank complaint comment or reason, at most 2000 Unicode characters.
recorded_at	timestamp	NO	-	Stored submission/note/review/audit recording time.

Kinds COMMENT, ACCEPT_REASON, REJECT_REASON, WITHDRAW_REASON, REOPEN_REASON. Valid posting roles and nonblank body up to 2000 Unicode characters. Store author ID/role; name is read from users. Runtime SELECT/INSERT only.

Reported defaults: note_id = <none> (auto increment); recorded_at = CURRENT_TIMESTAMP.

asset inspections

Scheduled/assigned inspections and retained pass/fail findings and dates.

Field	SQL type	Null	Key	Meaning
inspection_id	int	NO	PK	Primary identifier.
asset_id	int	NO	FK	Related asset reference.
inspection_type	varchar(150)	NO	-	Inspection task/type description.
scheduled_date	date	NO	-	Planned inspection date.
status	varchar(20)	NO	-	Current workflow/availability state; see table rules.
inspected_by_user_id	int	YES	FK	Actual completing inspector; null while scheduled.
notes	text	YES	-	Completed inspection findings.
completed_date	date	YES	-	Inspection completion date; distinct from recording timestamp.
assigned_to_user_id	int	YES	FK	Assigned inspector; nullable for unassigned scheduled inspections.
completed_at	datetime(6)	YES	-	Precise inspection result recording time; separate from completed_date.

SCHEDULED has no completing inspector/date. PASSED/FAILED require inspector, notes and completion date; nonblank findings/assignment/date policy is checked by service. Failed completion creates a hold atomically.

Reported defaults: inspection_id = <none> (auto increment); status = SCHEDULED.

preventive maintenance schedules

Recurring active/paused tasks with frequency and next due date; no completion-history rows.

Field	SQL type	Null	Key	Meaning
maintenance_schedule_id	int	NO	PK	Primary identifier.
asset_id	int	NO	FK	Related asset reference.
task_description	varchar(255)	NO	-	Recurring maintenance task description.
frequency_days	int	NO	-	Positive recurrence interval in days.
next_due_date	date	NO	-	Current due date; completion advances it by frequency.
status	varchar(20)	NO	-	Current workflow/availability state; see table rules.

ACTIVE or PAUSED, frequency_days > 0. Completion updates next_due_date; no preventive work order/history table.

Reported defaults: maintenance_schedule_id = <none> (auto increment); status = ACTIVE.

asset safety holds

Individual failed-inspection holds and explicit evidence-backed administrator clearance.

Field	SQL type	Null	Key	Meaning
hold_id	int	NO	PK	Primary identifier.
asset_id	int	NO	FK	Related asset reference.
failed_inspection_id	int	NO	FK, UQ member	Unique originating failed inspection; each failure has its own hold.
created_by_user_id	int	NO	FK	User who recorded the failed inspection/hold.
created_at	datetime(6)	NO	-	Hold recording time.
cleared_by_user_id	int	YES	FK	Administrator who cleared this hold; null while active.
clearance_reason	varchar(1000)	YES	-	Required explicit clearance explanation; null while active.
passing_inspection_id	int	YES	FK	Optional chosen later passing inspection evidence.
verified_work_order_id	int	YES	FK	Optional chosen verified corrective repair evidence.
cleared_at	datetime(6)	YES	-	Recorded clearance time; null while active.

An active hold has no clearance fields. Cleared hold requires administrator/reason/time and exactly one evidence type. Temporal/same-asset evidence validity is rechecked by service.

Reported defaults: hold_id = <none> (auto increment); created_at = CURRENT_TIMESTAMP(6).

Unique tuple: failed_inspection_id.

asset lifecycle history

Recorded manual lifecycle transitions with actor, reason and time.

Field	SQL type	Null	Key	Meaning
history_id	int	NO	PK	Primary identifier.
asset_id	int	NO	FK	Related asset reference.
actor_user_id	int	NO	FK	Acting administrator reference.
previous_lifecycle	varchar(30)	NO	-	Lifecycle before the recorded transition.
next_lifecycle	varchar(30)	NO	-	Lifecycle after the recorded transition.
reason	varchar(1000)	NO	-	Required manual lifecycle change explanation.
changed_at	datetime(6)	NO	-	Manual lifecycle transition recording time.

Actor/reason/time preserve manual transitions. Runtime table grants do not by themselves guarantee append-only history; application writes insert transitions.

Reported defaults: history_id = <none> (auto increment); changed_at = CURRENT_TIMESTAMP(6).

action audit

Application/operator events. Interpret target_id with action; it has no generic target foreign key.

Field	SQL type	Null	Key	Meaning
audit_id	bigint	NO	PK	Primary identifier.
actor_user_id	int	YES	FK	Acting user reference; nullable audit actor can represent operator activity.
action	varchar(80)	NO	-	Recorded event name.
target_id	int	YES	-	Event-specific target identifier; pair with action.
recorded_at	timestamp	NO	-	Stored submission/note/review/audit recording time.

Recorded events are written with their operation transaction. target_id is intentionally not a foreign key because targets have different entity types.

Reported defaults: audit_id = <none> (auto increment); recorded_at = CURRENT_TIMESTAMP.

security gate

Single-row local coordination gate, not a user-facing work item.

Field	SQL type	Null	Key	Meaning
gate_id	int	NO	PK	Single coordination-row key, value 1.

The current application serializes local service operations through row 1.

operator setup

Operator-only coordination row for deliberate bootstrap/recovery.

Field	SQL type	Null	Key	Meaning
setup_id	int	NO	PK	Single operator coordination-row key, value 1.

No runtime permission. Operator bootstrap/recovery uses this coordination row.

schema migrations

Operator-managed migration version/checksum/STARTED or COMPLETE state; runtime reads only.

Field	SQL type	Null	Key	Meaning
version	int	NO	PK	Numbered migration version.
checksum	char(64)	NO	-	Canonical resource SHA-256.
status	varchar(20)	NO	-	STARTED or COMPLETE operator migration state.

Runtime checks versions 1-5 COMPLETE and matching resource checksums plus schema definitions. No automatic migration at sign-in.